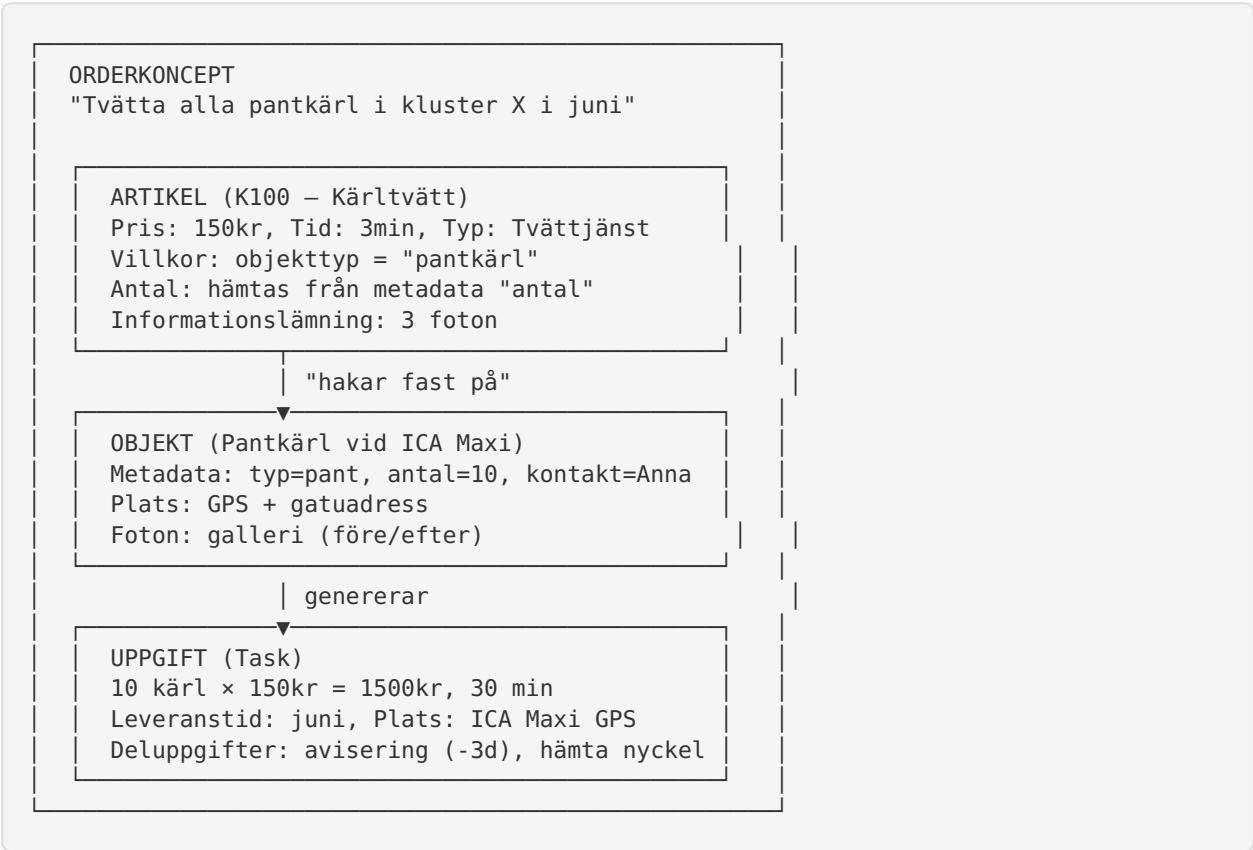


Traivo — Konsoliderad Replit-specifikation: Objekt- & Artikelhantering

Version: 1.0 — Master Summary
Datum: 2026-06-10
Baserat på: Tre sessioner (2026-06-09 × 1, 2026-06-10 × 2)
Syfte: EN komplett rapport som täcker allt Replit-teamet behöver för att implementera objekt- och artikelhantering i Traivo

Executive Summary

Traivo är ett system för att hantera **tjänster och varor** kopplade till **fysiska eller organisatoriska objekt** (t.ex. avfallskärl, fastigheter, återvinningsrum). Systemets kärna kan beskrivas i fyra lager:



Ingen bakåtkompatibilitet krävs — detta är version 1 av systemet.

1. Objekt-hantering

1.1 Vad är ett objekt?

Ett objekt är **allt fysiskt eller organisatoriskt i kundens kontext** — avfallskär!, återvinningsrum, fastigheter, organisationer. Objekt bildar en **hierarki** (förälder → barn):

```

Organisation (BRF Solgläntan)
├── Fastighet (Storgatan 12)
│   ├── Återvinningsrum (Rum A)
│   │   ├── Pantkärl (10 st)
│   │   └── Matavfallskärl (5 st)

```

1.2 Objektets identitet

Egenskap	Beskrivning
Namn	Fritt namn, t.ex. "Rum A, Storgatan 12"
Identitet	Unik identitet per objekt
Systemlöpnummer	Organisatoriskt, automatgenererat
Förälderkoppling	Referens till överordnat objekt i hierarkin

1.3 Metadata — "Post-it-lappar på objektet"

Metadata är **nyckel-värde-par** som beskriver objektets attribut. Varje metadatafält har en definierad datatyp och beteende.

Metadatatyper (dataformat)

Datatyp	Exempel	Beskrivning
Alfanumeriskt	"Anna Svensson"	Fri text
Numeriskt	10	Heltal/decimaltal
Fördefinierade val	"röd" \ "blå" \ "grön"	Dropdown/enum
Status	"okej" \ "trasig"	Binärt eller flerväls-status
Filreferens	foto_001.jpg	Foto eller uppladdad fil
Grupp	{namn, titel, epost, telefon}	Sammanfattat fält (t.ex. kontakt)
Geografisk	{gata, postadress, postort, GPS_x, GPS_y, what3words}	Platsinformation

Konkreta metadataexempel på ett objekt

```
{
  "objekttyp": { "value": "pant", "datatype": "predefined", "options": ["pant", "matavfall", "återvinning"] },
  "antal": { "value": 10, "datatype": "numeric" },
  "kontakt": {
    "datatype": "group",
    "fields": {
      "namn": "Anna Svensson",
      "titel": "Fastighetsskötare",
      "epost": "anna@brfsolgläntan.se",
      "telefon": "070-1234567"
    }
  },
  "bilddokumentation_fore": { "value": "foto_001.jpg", "datatype": "file", "displayMode": "gallery" },
  "bilddokumentation_efter": { "value": null, "datatype": "file", "displayMode": "gallery" },
  "plats": {
    "datatype": "geo",
    "gata": "Storgatan 12",
    "postadress": "123 45",
    "postort": "Stockholm",
    "gps_x": 59.3293,
    "gps_y": 18.0686,
    "what3words": "filled.count.soap"
  }
}
```

1.4 Arvsregler för metadata

Kritisk logik — avgör vilken data som “faller nedåt” i hierarkin:

Metadatafält	Ärver nedåt?	Motivering
Kontaktperson	✓ Ja	Samma kontakt gäller alla kärl i rummet
Plats/Geografi	✓ Ja	Alla kärl i rummet har samma adress
Antal	✗ Nej	Varje objekt har sitt eget antal
Foto	✗ Nej	Foto av rummet ≠ foto av kärlet inuti
Objekttyp	✗ Nej (kontextberoende)	Varje nivå har sin egen typ

Implementation: Varje metadatadefinition ska ha en boolean `inheritsDown` som styr arv.

1.5 Spårbarhet (audit trail)

Varje gång ett metadatafält uppdateras ska det taggas med:

Tagg	Beskrivning
Tidpunkt	Exakt timestamp för uppdateringen
Användare	Vem som gjorde ändringen
Uppgiftsnummer	Om uppdateringen skedde inom en uppgift
Källa	system extern_import manuell_operatör

1.6 Uppdateringslogik per metadatatyp

Typ	Vid uppdatering	Resultat
Numeriskt (antal)	Nytt värde ersätter gammalt	Gammalt arkiveras som <code>is-Active: false</code>
Kontakt (grupp)	Ny kontakt läggs till	Gallerivvy — alla syns, bläddra mellan dem
Foto (fil)	Nytt foto läggs till	Fotogalleri sorterat efter datum
Geografisk (plats)	Ny position ersätter	Bara ett aktivt värde, systemet väljer mest "högupplösta" varianten

1.7 Geografisk hantering — speciella regler

- Systemet ska **geokoda** automatiskt och välja den mest preciserade varianten
- Prioritering: GPS-koordinater > gatuadress > postort
- **Operatörer** kan uppdatera geografi via gatuadress ELLER kartreferens (geotagging)
- Platsinformation ärvs nedåt i hierarkin

2. Artikel-hantering

2.1 De sju informationsfälten

Sessionen definierar att en artikel har **exakt 7 informationskapitel**:

#	Kapitel	Innehåll
1	Grunduppgifter	Artikelnummer (Fortnox API), beskrivning, artikeltyp
2	Fördjupad information	Filer, instruktioner, leverantörsinfo, extern länk
3	Priser, tider & plats	Listpris, kostnad, produktionstid, offsettid, leddid, geokoppling, lagerplats, strukturartikel
4	Haka på information	Metadatatavillkor (association/tvåstegsfilter)
5	Artikels antal	Fast antal, dynamiskt från metadata, eller manuellt i orderkoncept
6	Informationslämning	Metadata som ska uppdateras vid utförande (foton, status, antal)
7	Utförandetyp / tjänstetyp	Kodifiering för sortering och tilldelning av uppgifter

2.2 Kapitel 1: Grunduppgifter

Fält	Typ	Källa	Beskrivning
Artikelnummer	Sökbar dropdown	Fortnox API	Unikt per organisation. Söks via API mot Fortnox
Artikelbeskrivning	Text (max 50 tkn)	Fortnox API	Syns på följesedlar, fakturor, kundkorrespondens. Begränsad längd pga fakturaformat
Intern beskrivning	Textarea	Manuell	Fördjupande info, ej synlig för kund. T.ex. "Spola särskilt i hörnen, ta tre foton höger→vänster"
Artikeltyp	Dropdown (bibliotek)	Admin-register	Se §2.3

2.3 Artikeltyper (administrerbart bibliotek)

Typ	Kategori	Exempel
Tvättjänst	Tjänst	Kärltvätt
Besiktningstjänst	Tjänst	Kontroll av lyftkrok
Torrtjänst	Tjänst	Torrtvätt
Kontroll	Tjänst	Statuscheck med sant/falskt
Felanmälan	Tjänst	Felanmälan/reparation
Reservdelar	Vara	Skruvar, klistermärken
Förbrukningsvaror	Vara	Rengöringsmedel

Krav: Registret är **dynamiskt** — admin skapar/redigerar typer utan kodsläpp.

2.4 Kapitel 2: Fördjupad information

Fält	Typ	Beskrivning
Extern info — länk	URL	Länk till produktblad, kvalitetssystem etc.
Länkbeskrivning	Textarea	Beskrivning av den externa länken
Filuppladdning	File input	PDF, bild — instruktion, utförandebeskrivning
Leverantörsnummer	Tag-input (multi)	Externa artikelnummer hos leverantörer
Leverantörsnamn	Text	Leverantörens namn

2.5 Kapitel 3: Priser, tider & plats

Ekonomi

Fält	Typ	Default	Beskrivning
Listpris	Numeriskt (kr)	0.00	Pris mot kund per enhet
Kostnad	Numeriskt (kr)	0.00	Inköps-/produktion-skostnad per enhet
Marginal	Beräknat	auto	listpris - kostnad (ej redigerbart)
Enhet	Text	st	Måttenhet (styck, timme, etc.)

Tider

Fält	Typ	Beskrivning
Produktionstid	Numeriskt (min)	Hur lång tid utförandet tar. Kritiskt för produktionsplanering
Offsettid	Enhet + värde + riktning	Tid relativt huvudjobb: före (negativ), samtidigt (0), efter (positiv). Enhet: minuter eller dagar
Ledtid	Numeriskt (dagar), nullable	Leveranstid för varor. Triggas beställning i förväg. null = ej relevant

Plats & geografi

Tre typer av geografiskt beroende:

Typ	Exempel	Hantering
Objektets plats	Tvätta kärl på Storgatan 12	Ärver GPS/adress från objektets metadata
Lagerplats	Hämta nyckel från lagret	En eller flera lagerplatser anges på artikeln
Ej geografiskt bunden	Telefonavisering	Ingen platskoppling

Geokoppling: Systemet väljer automatiskt den mest preciserade geografiska varianten. Prioritetsordning: GPS-koordinater > gatuadress > postort.

Strukturartikel

Fält	Typ	Default	Beskrivning
Är strukturartikel	Checkbox	false	Artikeln består av flera underartiklar (BOM)
Kan användas som komponent	Checkbox	true	Artikeln kan ingå i andra strukturartiklar
Beroendeartikel	Checkbox	false	Måste utföras/kvitteras innan huvuduppgift
Tillåt gruppavbockning	Checkbox	(beslut krävs)	Alla delartiklar kan klarmarkeras i grupp

Arv i strukturartiklar: Underartiklar ärver leveranstid, plats och antal från huvudartikel — om de inte har egna värden.

2.6 Kapitel 4: Haka på information (metadatavillkor / association)

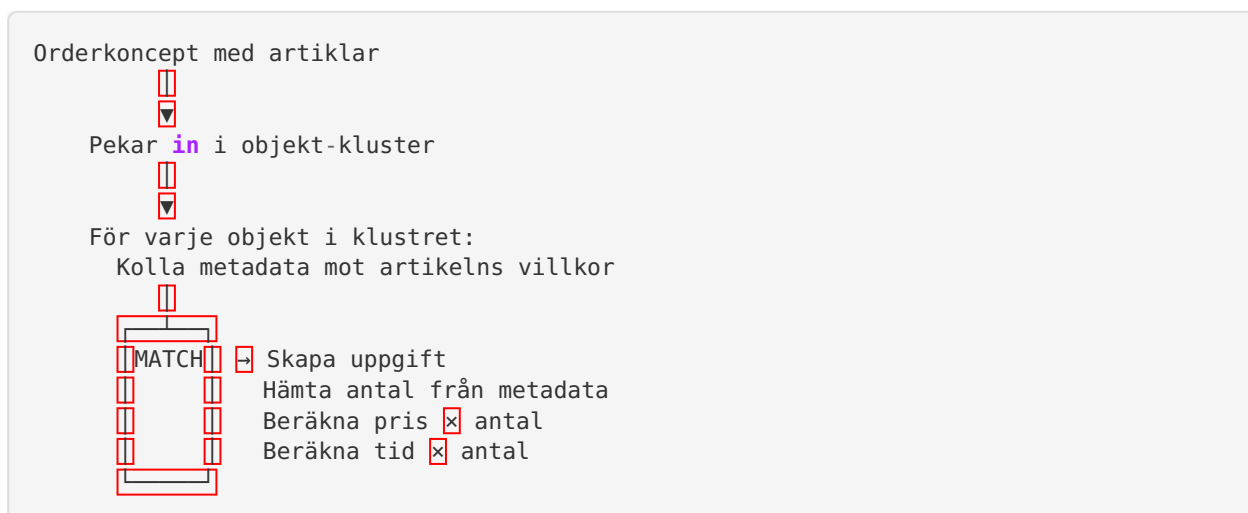
Avgör **vilka objekt** artikeln "hakar fast på" baserat på objektets metadata.

Villkorsoperatorer

Operator	Typ	Exempel
Lika med	Exakt match	objekttyp = "pantkärl"
Innehåller	Delsträng	kontakt.namn CONTAINS "Anna"
Större än	Numerisk	antal > 200
Mindre än	Numerisk	antal < 200
Skilt från	Negation	status != "trasig"

Logik: Flera villkor = **AND (OCH-logik)**. Alla villkor måste uppfyllas för träff.

Flöde vid orderkoncept-expansion



Max antal per adress

Fält	Typ	Default	Beskrivning
Max antal per adress	Numeriskt, nullable	null (obegränsat)	Begränsar t.ex. etablering till 1 gång per adress

2.7 Kapitel 5: Artikelns antal

Tre alternativ för hur antal bestäms:

Alternativ	Beteende	Exempel
Fast antal	Fördefinierat värde per träff	“Alltid 1 fotodokumentation per objekt”
Dynamiskt från metadata	Hämtas från valt metadatafält	“antal_kärl” → 10
Matas in i orderkoncept	Manuellt vid orderläggning	Operatören anger antal

Behörighetsstyrning för antal

Inställning	Effekt
Fri uppdatering tillåten	Operatören kan ändra antal → tid & pris uppdateras → metadatafältet på objektet uppdateras
Fri uppdatering ej tillåten	Operatören ser antal men kan inte ändra

2.8 Kapitel 6: Informationslämning

Definierar **vilken metadata som ska uppdateras** vid utförande. Flexibelt — kan vara flera fält.

Lämna-typ	Beteende	Exempel
Antal (kvantitativt)	Uppdatera metadatafält	“Tvättade 12 kärl” (uppdaterar antal)
Foto	Lägg till i galleri	“Foto vänster, mitt, höger” (3 metadatafält)
Statusval	Dropdown-val	“OK / Ej OK”
Fritext	Textfält	“Hörn behöver extra rengöring”

Slutförandekrav: Operatören **kan inte slutföra uppgiften** förrän alla konfigurerade informationslämningsfält är ifyllda (t.ex. alla 3 foton tagna).

2.9 Kapitel 7: Utförandetyp / tjänstetyp

Fält	Typ	Källa
Utförandetyp	Dropdown (bibliotek)	Admin-register

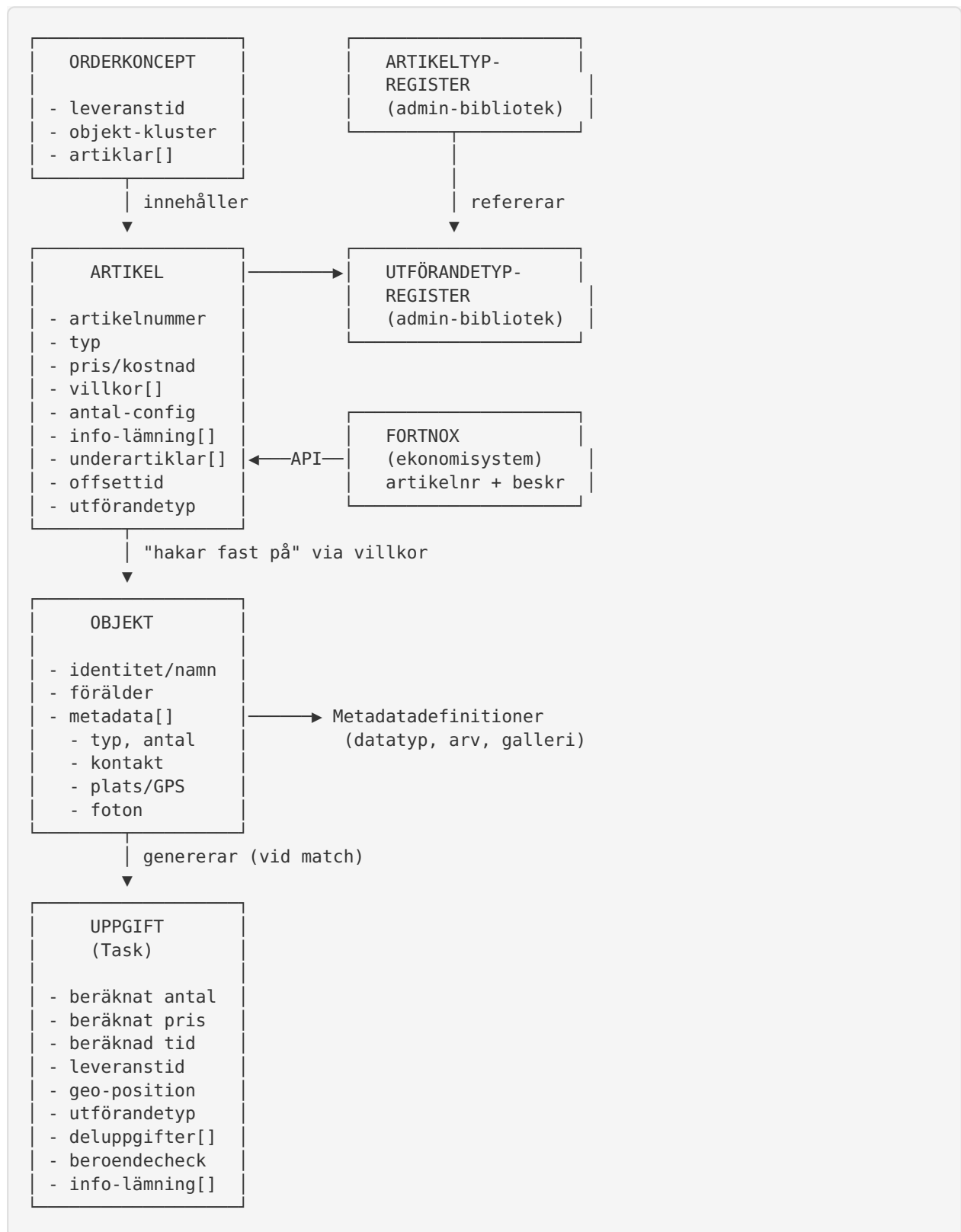
Syfte: Separat kodifiering (ej samma som artikeltyp!) för att:

- Sortera uppgiftslistor (alla “bottenöppnande” uppgifter på ett ställe)
- Tilldela uppgifter till rätt team/person
- Filtrera i grovplanering

Exempel: **RBK** (rullbara kärl), **ADM** (administrativa), **BOT** (bottenöppnande)

3. Relationer mellan entiteter

3.1 Entitetsdiagram



3.2 Dataflöde: Orderkoncept → Uppgift

1. **O**rderkoncept pekar in i objekt-kluster
 2. Varje artikel i **o**rderkonceptet evaluerar villkor mot varje objekt
 3. Vid träff: skapa uppgift
 - a. Hämta antal (fast / metadata / manuellt)
 - b. Beräkna: pris = **list**pris ☐ antal, kostnad = enhetskostnad ☐ antal, tid = produktionstid ☐ antal
 - c. Hämta leveranstid från **o**rderkoncept (eller metadata)
 - d. Hämta geografi från objekt (eller lagerplats, eller ingen)
 4. Kolla offsetartiklar ☐ skapa deluppgifter med tidsförskjutning
 5. Kolla beroendeartiklar ☐ flagga uppgifter som måste utföras i **o**rdning
 6. Uppgifter multipliceras per matchat objekt (kan bli hundratals)
 7. Uppgifter hamnar i grovplaneringslista ☐ finplanering ☐ ruttoptimering
-

4. Datamodeller (TypeScript interfaces)

4.1 Objekt

```

interface TraivoObject {
    id: string; // Systemlöpnummer (auto)
    name: string; // Fritt namn
    identity?: string; // Unik identitet
    parentId?: string; // Referens till förälderobjekt
    metadata: ObjectMetadata[]; // Alla metadatafält
    createdAt: string; // ISO timestamp
    updatedAt: string; // ISO timestamp
}

interface ObjectMetadata {
    id: string;
    definitionId: string; // Referens till MetadataDefinition
    value: MetadataValue; // Aktuellt värde
    isActive: boolean; // false = arkiverad
    history: MetadataAuditEntry[]; // Alla historiska ändringar
}

interface MetadataDefinition {
    id: string;
    name: string; // T.ex. "objekttyp", "antal", "kontakt"
    dataType: 'alphanumeric' | 'numeric' | 'predefined' | 'status' | 'file' | 'group' |
'geo';
    predefinedOptions?: string[]; // För 'predefined': ["röd", "blå", "grön"]
    groupFields?: string[]; // För 'group': ["namn", "titel", "epost",
"telefon"]
    inheritsDown: boolean; // Ärver nedåt i hierarkin
    displayMode: 'single' | 'gallery'; // 'single' = senaste ersätter, 'gallery' =
alla visas
}

type MetadataValue =
    | string // alphanumeric, predefined, status
    | number // numeric
    | FileReference // file
    | Record<string, string> // group
    | GeoPosition; // geo

interface GeoPosition {
    street?: string;
    postalCode?: string;
    city?: string;
    gpsX?: number; // latitude
    gpsY?: number; // longitude
    what3words?: string;
}

interface FileReference {
    fileId: string;
    fileName: string;
    mimeType: string;
    url: string;
    uploadedAt: string;
}

interface MetadataAuditEntry {
    previousValue: MetadataValue;
    newValue: MetadataValue;
    timestamp: string; // ISO
    userId: string; // Vem som ändrade
    taskId?: string; // Uppgiftsnummer om relevant
}

```

```
source: 'system' | 'external_import' | 'manual_operator';  
}
```

4.2 Artikel


```

interface Article {
  // — Kapitel 1: Grunduppgifter —
  articleNumber: string;           // Från Fortnox API, unikt per org
  articleDescription: string;      // Max 50 tecken, syns på faktura
  internalDescription?: string;    // Intern info för utförare
  articleType: string;             // Referens till artikeltyp-register

  // — Kapitel 2: Fördjupad information —
  externalInfoUrl?: string;
  externalInfoDescription?: string;
  externalInfoFile?: FileReference;
  supplierNumbers: string[];      // Externa leverantörsnummer
  supplierName?: string;

  // — Kapitel 3: Priser, tider & plats —
  listPriceKr: number;            // Default: 0.00
  costKr: number;                 // Default: 0.00
  // marginPerUnit: beräknas (listPriceKr - costKr)
  unit: string;                   // "st", "tim", etc.
  productionTimeMinutes?: number;

  offsetUnit: 'minutes' | 'days';
  offsetValue: number;            // Default: 0
  offsetType: 'before' | 'simultaneous' | 'after';

  leadTimeDays?: number | null;   // Leveranstid i dagar, null = ej relevant

  geoBindingType: 'object' | 'warehouse' | 'none';
  warehouseLocations?: GeoPosition[]; // Om geoBindingType = 'warehouse'

  isStructureArticle: boolean;    // BOM-artikel
  canBeComponent: boolean;        // Default: true
  subArticles?: SubArticleRef[];

  isDependencyArticle: boolean;   // Måste kvitteras innan huvuduppgift
  allowGroupCompletion?: boolean; // Tillåt gruppavbokning

  // — Kapitel 4: Haka på information —
  associationConditions: AssociationCondition[];
  maxPerAddress?: number | null;  // null = obegränsat
  defaultMetadataField?: string;  // "Hakar fast på" – förvalt fält

  // — Kapitel 5: Artikelns antal —
  quantityMode: 'fixed' | 'fromMetadata' | 'manualInOrder';
  fixedQuantity?: number;         // Om quantityMode = 'fixed'
  quantityMetadataField?: string; // Om quantityMode = 'fromMetadata'
  operatorCanUpdateQuantity: boolean;
  freeMetadataUpdate: boolean;    // Om operatören uppdaterar antal → uppdatera metadata på objekt

  // — Kapitel 6: Informationslämning —
  deliveryMetadataFields: DeliveryField[];

  // — Kapitel 7: Utförandetyp —
  executionTypeCode: string;      // Referens till utförandetyp-register

  // — System —
  status: 'active' | 'inactive';
  createdAt: string;
  updatedAt: string;
}

```

```
interface SubArticleRef {
  articleNumber: string;
  inheritsQuantityFromParent: boolean;
  order: number; // Utförandeordning
}

interface AssociationCondition {
  metadataFieldName: string; // T.ex. "objekttyp"
  operator: 'equals' | 'contains' | 'greaterThan' | 'lessThan' | 'notEqual';
  value: string | number; // T.ex. "pantkärl" eller 200
}

interface DeliveryField {
  metadataDefinitionId: string; // Vilket metadatafält som ska uppdateras
  required: boolean; // Måste fyllas i för att slutföra uppgift
  order: number; // Visningsordning
}
```

4.3 Uppgift (Task)

```

interface Task {
  id: string;
  taskNumber: string; // Unikt uppgiftsnummer
  orderConceptId: string;
  articleNumber: string;
  objectId: string; // Matchat objekt

  // Beräknade värden
  quantity: number;
  totalPrice: number; // listPris × quantity
  totalCost: number; // kostnad × quantity
  estimatedTimeMinutes: number; // produktionstid × quantity

  // Tid
  desiredDeliveryDate?: string; // Från orderkoncept
  possibleDeliveryDate?: string; // Beräknad med hänsyn till offset/ledtid
  offsetDate?: string; // Om offset finns

  // Plats
  geoPosition?: GeoPosition; // Från objekt, lager, eller null
  geoBindingType: 'object' | 'warehouse' | 'none';

  // Utförande
  executionTypeCode: string;
  status: 'planned' | 'assigned' | 'in_progress' | 'completed' | 'warning';
  assignedUnitId?: string;

  // Beroenden & struktur
  parentTaskId?: string; // Om deltask i strukturartikel
  subTasks: Task[];
  isDependency: boolean;
  dependencyWarnings: DependencyWarning[];

  // Informationslämning
  deliveryFields: TaskDeliveryField[];
  completionRequirements: CompletionRequirement[];
}

interface TaskDeliveryField {
  metadataDefinitionId: string;
  expectedValue?: MetadataValue; // Hämtat från objekt
  actualValue?: MetadataValue; // Ifyllt av operatör
  isCompleted: boolean;
}

interface DependencyWarning {
  type: 'not_completed' | 'time_exceeded' | 'geo_mismatch';
  message: string; // T.ex. "Avisering ej utförd – 3 dagar kvar"
  relatedTaskId: string;
}

interface CompletionRequirement {
  type: 'quantity_confirmed' | 'photo_taken' | 'status_selected' | 'text_entered';
  metadataFieldId: string;
  isFulfilled: boolean;
}

```

4.4 Admin-register

```
interface ArticleTypeRegister {
  id: string;
  name: string;                                // "Tvättjänst", "Reservdelar"
  category: 'service' | 'product';
  isActive: boolean;
}

interface ExecutionTypeRegister {
  id: string;
  code: string;                                // "RBK", "ADM", "BOT"
  name: string;                                // "Rullbara kärl", "Administrativa"
  isActive: boolean;
}
```

5. API Endpoints

5.1 Objekt

Metod	Endpoint	Beskrivning
GET	/api/objects	Lista objekt (med hierarki-fil-ter)
GET	/api/objects/:id	Hämta objekt med all metadata
POST	/api/objects	Skapa nytt objekt
PATCH	/api/objects/:id	Uppdatera objekt
GET	/api/objects/:id/metadata	Hämta metadata med historik
PUT	/api/objects/:id/ metadata/:fieldId	Uppdatera metadatafält (med audit)
GET	/api/objects/:id/children	Hämta barnobjekt

5.2 Metadata-definitioner

Metod	Endpoint	Beskrivning
GET	/api/metadata-definitions	Lista alla metadatadefinitioner
POST	/api/metadata-definitions	Skapa ny definition
PATCH	/api/metadata-definitions/:id	Uppdatera (inheritsDown, displayMode etc.)

5.3 Artiklar

Metod	Endpoint	Beskrivning
GET	/api/articles	Lista artiklar (filter: typ, status)
GET	/api/articles/:id	Hämta komplett artikel
POST	/api/articles	Skapa ny artikel
PATCH	/api/articles/:id	Uppdatera artikel
POST	/api/articles/:id/upload	Ladda upp fil (PDF/bild)
GET	/api/articles/:id/sub-articles	Hämta underartiklar (BOM)

5.4 Admin-register

Metod	Endpoint	Beskrivning
GET	/api/registers/article-types	Lista artikeltyper
POST	/api/registers/article-types	Skapa ny artikeltyp
GET	/api/registers/execution-types	Lista utförandetyper
POST	/api/registers/execution-types	Skapa ny utförandetyp

5.5 Uppgifter

Metod	Endpoint	Beskrivning
GET	<code>/api/tasks</code>	Lista uppgifter (filter: status, utförandety, datum, geo)
GET	<code>/api/tasks/:id</code>	Hämta uppgift med deluppgifter
PATCH	<code>/api/tasks/:id/quantity</code>	Operatör uppdaterar antal
POST	<code>/api/tasks/:id/deliver</code>	Lämna metadata (foto, status, etc.)
POST	<code>/api/tasks/:id/complete</code>	Slutför uppgift (validerar krav)
GET	<code>/api/tasks/:id/warnings</code>	Hämta varningar (beroenden etc.)

5.6 Extern integration

Metod	Endpoint	Beskrivning
GET	<code>/api/fortnox/articles?search=K100</code>	Sök artiklar i Fortnox
POST	<code>/api/geocode</code>	Geokoda adress → koordinater

6. Kritiska affärsregler

6.1 Villkorslogik

1. **AND-logik:** Alla associationsvillkor måste uppfyllas för att artikel ska haka fast på objekt
2. **Multiplicering:** En artikel i ett orderkoncept kan skapa hundratals uppgifter (en per matchat objekt)
3. **Villkorsoperatorer:** `=`, `!=`, `>`, `<`, `CONTAINS` — alla agerar på metadatavärden

6.2 Antalshantering

1. **Dynamiskt antal:** Om `quantityMode = 'fromMetadata'` → hämta värde från angivet metadatafält
2. **Operatörsuppdatering:** Om tillåtet → uppdatera antal → automatisk omräkning av pris och tid → om `freeMetadataUpdate = true` → uppdatera även metadatafältet på objektet
3. **Beräkning:** `totalPris = listpris × antal`, `totalKostnad = kostnad × antal`, `totalTid = produktionstid × antal`

6.3 Beroenden & varningar

1. **Beroendeartiklar** måste kvitteras innan huvuduppgift kan utföras
2. **Offsetberäkning:** `offsetDatum = leveransdatum + offsetVärde` (negativt = före, positivt = efter)
3. **Ledtidsvarning:** Om `leveransdatum - ledtidDagar < idag` → varna
4. **Utförandeordning:** Artiklar utförs i den ordning de framställs i orderkonceptet

6.4 Slutförande

1. **Obligatorisk informationslämning:** Alla konfigurerade delivery-fält (foton, status) måste vara ifyllda
2. **Kvittens:** Operatören bekräftar att planerat antal är utfört (eller uppdaterar)
3. **Gruppavbockning:** Valfritt — kan kräva att varje delmoment bockas av individuellt

6.5 Metadata-arv

1. **Arvsregler:** Metadata med `inheritsDown = true` propageras automatiskt till barnobjekt
2. **Arkivering:** Numeriska metadatafält arkiveras vid uppdatering; grupper/foton har galleriform
3. **Geografi:** Bara en aktiv geo-position per objekt, systemet väljer mest preciserade

6.6 Geografisk planering

1. **Tre geo-lägen:** Artiklar är bundna till objektets plats, lagerplats, eller ej geografiskt bunden
2. **Lagerplatslogik:** Om artikel kräver vara från lager → skapa deluppgift att hämta på lagret med "minustid"
3. **Ruttoptimering:** Uppgifter kan ruttoptimeras i finplanering baserat på geo-position och utförandetyd

7. Implementation Priorities

Fas 1 — Kärnfunktionalitet (Prio 1)

#	Feature	Motivering
1	Objektmodell med hierarki	Grunden för allt — objekt, förälder-barn, metadata
2	Metadatadefinitioner + CRUD	Datatyp, arv, galleri/single — konfigurerbara
3	Artikelformulär (accordion, 6 sektioner)	Redan designat i UI
4	Fortnox API-integration	Artikelnummer + beskrivning
5	Associationsvillkor (AND-logik)	Koppling artikel → objekt
6	Kvantitetsberäkning	Fast/dynamisk/manuell med prisberäkning

Fas 2 — Uppgiftshantering (Prio 2)

#	Feature	Motivering
7	Uppgiftsgenerering	Orderkoncept → matchning → uppgifter
8	Offset- och ledtidsberäkning	Tidsplanering och varningar
9	Beroendeartiklar + varningssystem	Kvalitetssäkring
10	Operatörsvy med informationslämning	Fältarbetarens gränssnitt
11	Audit trail (metadata-spårbarhet)	Regelbunden spårning

Fas 3 — Avancerat (Prio 3)

#	Feature	Motivering
12	Strukturartiklar (BOM)	Underartiklar, arv, gruppavböckning
13	Grovplanering + finplanering	Listvy → ruttoptimering
14	Admin-register (artikeltyper, utförandetyper)	Dynamiska bibliotek
15	Geografisk planering + lagerplatslogik	Lagerplats, geokodning, "minustid"
16	Leverantörsspecifika kostnader	Ej verksamhetskritiskt v1, men "ta höjd"

8. Tekniska rekommendationer

8.1 UI/UX

- **Accordion-layout** för artikelformuläret (6 sektioner). Grundinformation öppen som default.
- **Dark mode** — Befintlig design använder mörkt tema.
- **Teckenräknare** på namnfält (0/50). Max 50 tecken bekräftat.
- **Dynamisk villkors-builder** för association — `[+ Lägg till villkor]` med metadatafält-drop-down + operator + värde.
- **Fortnox-sökning** — sökbar dropdown med autocomplete mot Fortnox API.

8.2 Databas

- **Metadata som JSONB** (PostgreSQL) eller liknande dokumentformat — flexibelt, sökbart.
- **Hierarkisk modell** med `parentId` + materialized path eller closure table för effektiva queries.
- **Historiktabell** för metadata-audit (append-only).
- **Geospatial index** på GPS-koordinater för ruttoptimering.

8.3 Beräkningslogik

```
// Prisberäkning vid uppgiftsgenerering
function calculateTaskEconomics(article: Article, quantity: number) {
  return {
    totalPrice: article.listPriceKr * quantity,
    totalCost: article.costKr * quantity,
    margin: (article.listPriceKr - article.costKr) * quantity,
    estimatedMinutes: (article.productionTimeMinutes ?? 0) * quantity,
  };
}

// Offsetdatum-beräkning
function calculateOffsetDate(mainDate: Date, article: Article): Date | null {
  if (article.offsetType === 'simultaneous') return mainDate;
  const offset = article.offsetUnit === 'days'
    ? article.offsetValue * 24 * 60
    : article.offsetValue;
  const direction = article.offsetType === 'before' ? -1 : 1;
  return new Date(mainDate.getTime() + direction * offset * 60 * 1000);
}

// Villkorsmatchning
function matchConditions(object: TraivoObject, conditions: AssociationCondition[]): boolean {
  return conditions.every(cond => {
    const meta = object.metadata.find(m => m.definitionId === cond.metadataFieldName &
    & m.isActive);
    if (!meta) return false;
    switch (cond.operator) {
      case 'equals': return meta.value == cond.value;
      case 'notEqual': return meta.value != cond.value;
      case 'contains': return String(meta.value).includes(String(cond.value));
      case 'greaterThan': return Number(meta.value) > Number(cond.value);
      case 'lessThan': return Number(meta.value) < Number(cond.value);
    }
  });
}
```

8.4 Varningssystem

```
interface WarningCheck {
  type: string;
  check: (task: Task) => DependencyWarning | null;
}

const warningChecks: WarningCheck[] = [
  {
    type: 'dependency_not_completed',
    check: (task) => {
      const unfinished = task.subTasks.filter(st => st.isDependency && st.status !== '
completed');
      if (unfinished.length > 0) {
        return { type: 'not_completed', message: `${unfinished.length} beroendeupp-
gift(er) ej utförda`, relatedTaskId: unfinished[0].id };
      }
      return null;
    },
  },
  {
    type: 'lead_time_exceeded',
    check: (task) => {
      // Kolla om ledtid har passerat leveransdatum
      // ...
      return null;
    },
  },
  {
    type: 'geo_mismatch',
    check: (task) => {
      // Kolla om operatör har lämnat lagerplats utan att hämta vara
      // ...
      return null;
    },
  }
];
```

9. Komplettn genomgångsexempel

Scenario: Kärltvätt av 10 pantkär

Steg 1 — Objektet finns:

```
Objekt: "Pantkär ICA Maxi"
├─ objekttyp: "pant"
├─ antal: 10
├─ kontakt: Anna Svensson (070-1234567)
├─ foto_fore: [galleri med 2 bilder]
└─ plats: Storgatan 12, GPS: 59.329/18.068
```

Steg 2 — Artikel K100 definieras:

```

Artikel: K100 - Enkel kärltvätt
├─ Artikeltyp: Tvättjänst
├─ Listpris: 150 kr, Kostnad: 110 kr
├─ Produktionstid: 3 min
├─ Villkor: objekttyp = "pantkärll"
├─ Antal: hämta från metadata "antal"
├─ Operatör kan uppdatera: Ja (fri uppdatering)
├─ Informationslämning: 3 foton (vänster, mitt, höger)
├─ Offset-artikel: Avisering (-3 dagar, beroende)
├─ Utförandetyp: RBK (rullbara kärll)
└─ Strukturartikel: Nej

```

Steg 3 — Orderkoncept expanderar:

1. K100 pekar in i objekt-kluster
2. Villkor: objekttyp = "pantkärll"  MATCH på "Pantkärll ICA Maxi"
3. Antal hämtas: metadata.antal = 10
4. Beräkning:
 - Pris: 10  150 = 1 500 kr
 - Kostnad: 10  110 = 1 100 kr
 - Tid: 10  3 = 30 min
5. Offset-check: Avisering -3 dagar  skapa deluppgift
6. Lagerplats-check: Nyckel behövs  skapa deluppgift med minustid

Steg 4 — Uppgifter genererade:

Huvuduppgift: Kärlltvätt, ICA Maxi (juni)

-  Deluppgift 1: Avisering (-3 dagar) [BEROENDE]
-  Deluppgift 2: Hämta nyckel (lager, -30 min) [BEROENDE]
-  Info-krav: 3 foton (vänster/mitt/höger)

Steg 5 — Operatören ute i fält:

1. Avisering kvitterad  (3 dagar innan)
2. Nyckel hämtad  (30 min innan)
3. Anländer till ICA Maxi
4. Konstaterar 12 kärll (ej 10)  uppdaterar antal
 - Ny beräkning: 12  150 = 1 800 kr, 36 min
 - Metadata "antal" på objektet uppdateras: 10  12
5. Tvättar klart
6. Tar 3 foton 
7. Slutför uppgift 

10. Öppna frågor — Status och svar

Källa: Session 2026-06-10T12:48 — dedikerad genomgång av de 7 öppna frågorna

✓ BESVARADE

1. Geo-prioritering

Fråga: Hur hanteras geokodning och val av mest högupplösta koordinater tekniskt?

Svar: Systemet hanterar geo-information med varierande upplösning. Om bara orten är känd (t.ex. "Strömstad") är det den upplösning som gäller. Finns gatuadress blir det bättre. Finns exakta GPS-koordinater eller What3Words (3×3 meters ruta) får man högsta upplösning. Systemet ska alltid automatiskt välja den mest preciserade varianten. Dessutom ska operatören kunna uppdatera positionen via en **kartfunktion** — dra en pinpoint/flagga till exakt rätt plats så att geo-informationen uppdateras med hög upplösning.

Teknisk implementation:

- **Upplösningshierarki (prioritetsordning):** What3Words / GPS-koordinater > gatuadress > postort
- **Automatiskt val:** Vid flera tillgängliga geo-varianter, välj den med högst precision
- **Kartvy för operatör:** Draggable marker på karta (Leaflet/Mapbox) → sparar GPS-koordinater
- **Geokodning:** Automatisk forward geocoding vid adressinmatning, reverse geocoding vid kartplacering

API/Kod-påverkan:

```
// Utöka GeoPosition med upplösningsnivå
interface GeoPosition {
  street?: string;
  postalCode?: string;
  city?: string;
  gpsX?: number;
  gpsY?: number;
  what3words?: string;
  resolution: 'city' | 'street' | 'gps' | 'what3words'; // NY
}

// Resolutionsprioritering
function selectBestGeo(positions: GeoPosition[]): GeoPosition {
  const priority = { what3words: 4, gps: 3, street: 2, city: 1 };
  return positions.sort((a, b) => priority[b.resolution] - priority[a.resolution])[0];
}
```

- **Ny endpoint:** POST /api/objects/:id/geo-update — för kartbaserad uppdatering med audit trail
- **UI-komponent:** Kartvy med draggable marker + geocode-sökfält i objektformuläret

2. Strukturartikel-begränsningar

Fråga: Ska vissa artikeltyper uteslutas från att ingå i strukturartiklar?

Svar: Inga begränsningar. Alla artiklar ska kunna ingå i strukturartiklar. Nästlade strukturartiklar (strukturartikel inom strukturartikel) ska vara tillåtet. Om det uppstår systemmässiga begränsningar accepteras det, men det ska inte finnas designade begränsningar. **Dock:** Systemet måste kunna varna för **cirkelreferenser** och **deadlocks** i beroendekedjor — t.ex. om offsettider skapar rundgång som gör att uppgiften blir omöjlig att utföra. Sådana varningar ska visas i grovplaneringen.

Teknisk implementation:

- **Obegränsat djup** på nästling, men med cykeldetektering
- **Cirkelreferens-validering** vid sparande av strukturartikel (grafsökning, DFS)

- **Deadlock-detektion** vid grovplanering — kontrollera att offset-/beroendetider inte skapar omöjliga tidsfönster
- **Varningsnivåer:** INFO (djup nästling), WARNING (potentiell tidskrock), ERROR (cirkelreferens)

API/Kod-påverkan:

```
// Cirkelreferens-check vid sparande
function detectCircularReference(articleId: string, subArticles: SubArticleRef[]): boolean {
  const visited = new Set<string>();
  function dfs(id: string): boolean {
    if (visited.has(id)) return true; // Cirkel hittad
    visited.add(id);
    const subs = getSubArticles(id);
    return subs.some(sub => dfs(sub.articleNumber));
  }
  return dfs(articleId);
}

// Ny varningstyp
interface PlanningWarning {
  type: 'circular_reference' | 'offset_deadlock' | 'deep_nesting';
  severity: 'info' | 'warning' | 'error';
  articleIds: string[];
  message: string;
}
```

- **Ny endpoint:** POST /api/articles/:id/validate-structure — validerar BOM-träd före sparande
- **Grovplaneringsvy:** Visa varningar inline med röd/gul markering

3. Gruppavbockning vs steg-för-steg

Fråga: Hur ska fältarbetare markera artiklar som avklarade?

Svar: Varje uppgift har **kravmarkerade delmoment** (mandatory fields) som måste slutföras innan hela uppgiften kan kvitteras. Delmoment inkluderar: bekräfta/uppdatera antal, ta foto(n), ange status, osv. Varje ofullständigt obligatoriskt fält **lyser rött** i utförare-appen. Uppgiften kan inte slutföras förrän alla lyser grönt.

Gruppavbockning tillåts i standardiserade uppdrag — t.ex. 10 kärll med ett foto i slutet — där man kan klicka “klar” i ett svep, **förutsatt att alla delmoment redan är inlevererade**. Om ett foto saknas eller ett statusfält inte är ifyllt, blockeras gruppavbockning och det aktuella fältet markeras rött.

Teknisk implementation:

- **Per uppgift:** Array av `CompletionRequirement` med `isFulfilled: boolean`
- **Slutförandevalidering:** `canComplete = deliveryFields.every(f => f.isCompleted)`
- **Gruppavbockning:** Konfigureras per artikel via `allowGroupCompletion: boolean`
- **UX:** Rött = obligatoriskt men ofullständigt, grönt = klart, grått = valfritt
- **Gruppavbockning i praktiken:** En “Slutför alla”-knapp som validerar alla delmoment och bara blir klickbar när samtliga är gröna

API/Kod-påverkan:

```
// Slutförande-endpoint med validering
app.post('/api/tasks/:id/complete', (req, res) => {
  const task = getTask(req.params.id);
  const incomplete = task.deliveryFields.filter(f => f.required && !f.isCompleted);
  if (incomplete.length > 0) {
    return res.status(400).json({
      error: 'INCOMPLETE_REQUIREMENTS',
      fields: incomplete.map(f => f.metadataDefinitionId),
      message: `${incomplete.length} obligatoriskt fält är ej ifyllt`
    });
  }
  completeTask(task);
});
```

4. Dynamiskt antal — metadatafält

Fråga: Vilka numeriska fält ska visas i rullgardinen för dynamiskt antal?

Svar: Antal på uppgiften är **uppgiftsavgörande** — det styr pris, tid och kostnad. Det finns **tre alternativ**:

1. **Dynamiskt från metadata:** Välj ett metadatafält på objektet (mest troligt fältet "antal"). Dropdown visar tillgängliga numeriska metadatafält.
2. **Fast antal:** Hårdkodat på artikeln (t.ex. alltid 1 eller 10).
3. **Manuellt i orderkoncept:** Operatören anger antal vid orderläggning.

Hierarki för antal (mest sanna informationen vinner):

1. **Orderkoncept** (grova nivå) — default om inget annat finns
2. **Metadatafält på objekt** (finare nivå) — överridar orderkoncept
3. **Utförarens input på plats** (mest sanna) — överridar allt ovan

Det finns möjlighet att **låsa** antalet så att det inte kan ändras av operatör/utförare.

Teknisk implementation:

- **Dropdown-källa:** Filtrera `MetadataDefinition` där `dataType = 'numeric'` → visa i dropdown
- **Senaste värdet ersätter:** Antal är av typen `displayMode: 'single'` — ej galleri
- **Hierarki-engine:** Beräkna effektivt antal enligt prioritetsordning

API/Kod-påverkan:

```
// Hämta tillgängliga antal-fält för dropdown
app.get('/api/metadata-definitions?dataType=numeric', (req, res) => {
  // Returnerar alla numeriska metadata-fält för dropdown-val
});

// Hierarkisk antalsberäkning
function resolveQuantity(article: Article, object: TraivoObject, orderConceptQty?: number, performerQty?: number): number {
  // 1. Utförarens input (högst prioritet)
  if (performerQty !== undefined && article.operatorCanUpdateQuantity) return performerQty;
  // 2. Metadata-fält på objekt
  if (article.quantityMode === 'fromMetadata' && article.quantityMetadataField) {
    const meta = object.metadata.find(m => m.definitionId === article.quantityMetadataField && m.isActive);
    if (meta) return Number(meta.value);
  }
  // 3. Fast antal på artikel
  if (article.quantityMode === 'fixed' && article.fixedQuantity) return article.fixedQuantity;
  // 4. Orderkoncept
  if (orderConceptQty !== undefined) return orderConceptQty;
  return 1; // Fallback
}
```

- **Ny flagga i Article:** `quantityLocked: boolean` — om true, utförare kan ej ändra

5. Galleri vs statusfält

Fråga: Vem/vad bestämmer visningsformat för metadata?

Svar: Det bestäms i **metadatadefinitionen** i administrationen. Varje metadata-fält definieras med:

- **Namn** — vad fältet heter
- **Datotyp** — numeriskt, fördefinierat val, fil/foto, galleri, etc.
- **Visningsbeteende** — “single” (senaste ersätter tidigare) eller “gallery” (alla visas)

Exempel:

- **Numeriskt fält (antal):** `displayMode: 'single'` — senaste värdet överridar. Det är inte en bok med 8, 10, 12 — utan det senaste gäller.
- **Status (val):** `displayMode: 'single'` — “OK” eller “Ej OK”, senaste gäller.
- **Foto/fil:** `displayMode: 'gallery'` — alla foton visas i en gallerivy med kommentarer.
- **Kontakt (grupp):** `displayMode: 'gallery'` — alla kontakter visas, bläddra mellan dem.

Teknisk implementation: Redan korrekt modellerat i `MetadataDefinition.displayMode`. Inga ändringar i datamodell behövs.

API/Kod-påverkan:

```
// Uppdateringslogik styrd av displayMode
function updateMetadata(object: TraivoObject, fieldId: string, newValue:
MetadataValue) {
  const definition = getDefinition(fieldId);
  if (definition.displayMode === 'single') {
    // Arkivera gammalt värde, sätt nytt som aktivt
    deactivateOld(object, fieldId);
    addNew(object, fieldId, newValue, true);
  } else {
    // Gallery: lägg till utan att arkivera
    addNew(object, fieldId, newValue, true);
  }
}
```

- **Inga nya endpoints behövs** — beteendet styrs helt av befintlig `displayMode`-flagga i metadata-definitionen

6. Leveranstider från orderkoncept

Fråga: Ska detta vara perioder, scheman eller metadatabaserat?

Svar: Leveranstiden anges med **start-datum/tid** och **slut-datum/tid**. Tre format stöds:

Format	Exempel	Klockslag
Specifikt tidsfönster	Torsdag 3:e 10:00–12:00	Ja, kritiskt
Lång period	1 feb – 1 apr	Mindre viktigt
Periodiskt	Var 30:e dag ±5 dagar	Mindre viktigt

Parallella tidsfönster: Systemet stöder flera tidsfönster som gäller samtidigt. Exempel: leveransperiod feb–mars **OCH** torsdag eftermiddagar 12:00–16:00. Detta ger effektivt en intersection av tidsfönstren.

Teknisk implementation:


```

interface DeliveryTimeSpec {
    // Grundformat: alltid start + slut
    startDate: string;           // ISO datetime
    endDate: string;             // ISO datetime
    timeSignificant: boolean;    // Är klockslaget viktigt?

    // Periodisk leverans (valfritt)
    periodicIntervalDays?: number; // T.ex. 30
    periodicToleranceDays?: number; // T.ex. 5 (±)

    // Parallella tidsfönster (valfritt)
    additionalWindows?: TimeWindow[];
}

interface TimeWindow {
    dayOfWeek?: number;          // 0=mån, 6=sön (null = alla dagar)
    startTime: string;           // "12:00"
    endTime: string;             // "16:00"
}

```

API/Kod-påverkan:

- **Utöka orderkoncept-modell** med `deliveryTimeSpecs: DeliveryTimeSpec[]`
- **Beräkningslogik:** Vid periodisk leverans → generera tidsfönster automatiskt baserat på intervall + tolerans
- **Parallell intersection:** Om flera tidsfönster → beräkna effektiv leveranstid som intersection
- **UI:** Formulär med datumväljare + valfria tillägg: "Lägg till periodisk leverans" / "Lägg till parallellt tidsfönster"

VÄNTAR PÅ BESLUT

7. Varningströsklar

Fråga: Exakta värden för när leddidsvarningar ska triggas?

Kontext: Frågan ställdes direkt i sessionen. Svaret var: "Det vet inte jag just nu." Beställaren beskrev dock **grundlogiken**: Om en beroendeuppgift inte är utförd vid sin deadline ska systemet varna (röd markering eller alert). Exempel: Om huvuduppgift är planerad till torsdag 13:00 och beroendeuppgift ska vara klar 2 dagar innan → om beroendeuppgiften ej är kvitterad kl 12:00 på tisdag → varning. Geografiska beroenden (t.ex. nyckelplockning baserat på fysisk plats) nämndes som framtida komplexitet.

Rekommendation: Implementera ett **konfigurerbart tröskelsystem** med rimliga defaults:

Varningstyp	Default-tröskel	Konfigurerbar?	Beskrivning
Beroendeuppgift ej klar	0 min (omedelbart vid passerad deadline)	Ja	Röd markering när deadline passerat
Beroendeuppgift närmar sig	25% av återstående tid	Ja	Gul varning, t.ex. 12h kvar av 2 dagar
Ledtid vara	leveransdatum - ledtidDagar	Nej (beräknad)	Om idag > beställningsdatum → varning
Tidsfönster-konflikt	Vid planering	Nej (statisk)	Parallella tidsfönster som inte överlappar
Cirkelreferens	Vid sparande	Nej (direkt)	Omedelbar error vid detektering

Förslag till default-värden:

```
interface WarningThresholds {
  dependencyOverdueMinutes: number; // Default: 0 (omedelbar varning)
  dependencyApproachingPercent: number; // Default: 25 (gul vid 25% kvar)
  leadTimeBufferDays: number; // Default: 1 (varna 1 dag extra före)
  maxNestingDepthWarning: number; // Default: 5 (info vid djupare nästling)
}

const DEFAULT_THRESHOLDS: WarningThresholds = {
  dependencyOverdueMinutes: 0,
  dependencyApproachingPercent: 25,
  leadTimeBufferDays: 1,
  maxNestingDepthWarning: 5,
};
```

Teknisk rekommendation:

- Gör trösklar konfigurerbart per organisation i admin-inställningar
- Tre varningsnivåer:  OK →  Närmar sig deadline →  Passerat deadline
- Implementera som en **bakgrundsjobb** som kör var 15:e minut och uppdaterar varningsstatus
- **Ny endpoint:** GET /api/settings/warning-thresholds + PATCH /api/settings/warning-thresholds
- **Ny admin-vy:** Inställningssida för varningströsklar med slider/input per tröskeltyp

11. Sammanfattning av sessionsanalys

#	Fråga	Status	Sammanfattning
1	Geo-prioritering	✓ Besvarad	What3Words/GPS > gatuadress > postort. Kartvy med drag-gable marker.
2	Strukturartikel-be-gränsningar	✓ Besvarad	Inga begränsningar. Cykeldetektering + deadlock-varning.
3	Gruppavbockning vs steg-för-steg	✓ Besvarad	Kravmarkerade del-moment, rött/grönt. Gruppavbockning om alla delar klara.
4	Dynamiskt antal — metadatafält	✓ Besvarad	Dropdown med numeriska fält. Hierarki: orderkoncept → metadata → utförare.
5	Galleri vs statusfält	✓ Besvarad	Bestäms i metadata-definition (displayMode: single/gallery).
6	Leveranstider	✓ Besvarad	Start+slut datetime. Stöder perioder, periodiskt, parallella tidsfönster.
7	Varningströsklar	⌚ Väntar	Exakta värden ej beslutade. Konfigurerbart system rekommenderat.

Dokumentet konsoliderar fyra sessioners analys (2026-06-09 × 1, 2026-06-10 × 3). Avsett som master-referens för Replit-teamets implementationsarbete av Traivos objekt- och artikelhantering.